

BANK SYSTEM IN JAVA

Step-by-Step Execution Process, Algorithm, UML Class Diagram and Java Source Code

1. Classes Used

Class	Purpose
Account	Base class containing account number, balance, account type, deposit, withdrawal, transfer and display operations.
SavingsAccount	Derived class. Adds interest-rate data and an interest calculation method.
CurrentAccount	Derived class. Adds overdraft limit and overrides withdraw() to allow overdraft up to the limit.
BankSystem	Main class. Creates account objects, displays the menu and processes user choices.

2. Step-by-Step Execution Process

1. Program execution starts from main() in the BankSystem class.
2. A Scanner object is created to read input from the keyboard.
3. A SavingsAccount object is created with account number 101, balance 5000 and interest rate 5%. Its constructor calls super(), which initializes the Account fields and sets the type to Savings.
4. A CurrentAccount object is created with account number 201, balance 3000 and overdraft limit 2000. Its constructor calls super() and sets the type to Current.
5. The do-while loop displays the Bank Menu.
6. The user enters a choice. Scanner nextInt() reads the choice into ch.
7. The switch(ch) statement selects the required banking operation.
8. For Deposit (Savings), nextDouble() reads the amount and sAcc.deposit(amount) increases the savings balance.
9. For Withdraw (Savings), sAcc.withdraw(amount) calls the inherited Account.withdraw() method. Withdrawal is allowed only when the amount is not greater than the balance.
10. For Interest (Savings), sAcc.calcInterest() calculates interest using $\text{balance} \times \text{rate} / 100$. The interest is displayed but is not added to the balance.
11. For Withdraw (Current), cAcc.withdraw(amount) calls the overridden CurrentAccount.withdraw() method. It allows withdrawal up to balance + overdraft limit.
12. For Transfer Savings -> Current, sAcc.transfer(cAcc, amount) subtracts the amount from savings and adds it to the current account if sufficient savings balance exists.
13. For Show Accounts, show() displays the details of both accounts.

14. If the user enters 7, the program displays Bye! and the loop ends.

15. The Scanner is closed and the program terminates.

3. Algorithm

1. Start.

2. Create Scanner object.

3. Create SavingsAccount(101, 5000, 5).

4. Create CurrentAccount(201, 3000, 2000).

5. Repeat the following until the user chooses Exit:

6. a) Display the bank menu.

7. b) Read the user's choice.

8. c) If choice = 1, read amount and deposit into Savings Account.

9. d) If choice = 2, read amount and withdraw from Savings Account if sufficient balance exists.

10. e) If choice = 3, calculate Savings Account interest.

11. f) If choice = 4, read amount and withdraw from Current Account subject to overdraft limit.

12. g) If choice = 5, transfer amount from Savings Account to Current Account if sufficient balance exists.

13. h) If choice = 6, display both account details.

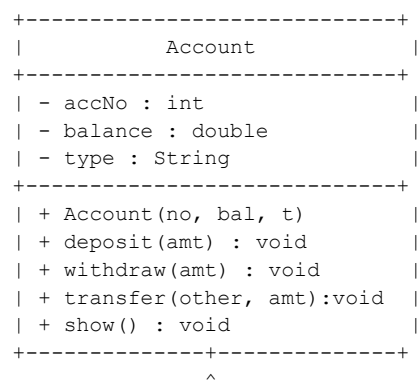
14. i) If choice = 7, display Bye! and stop the loop.

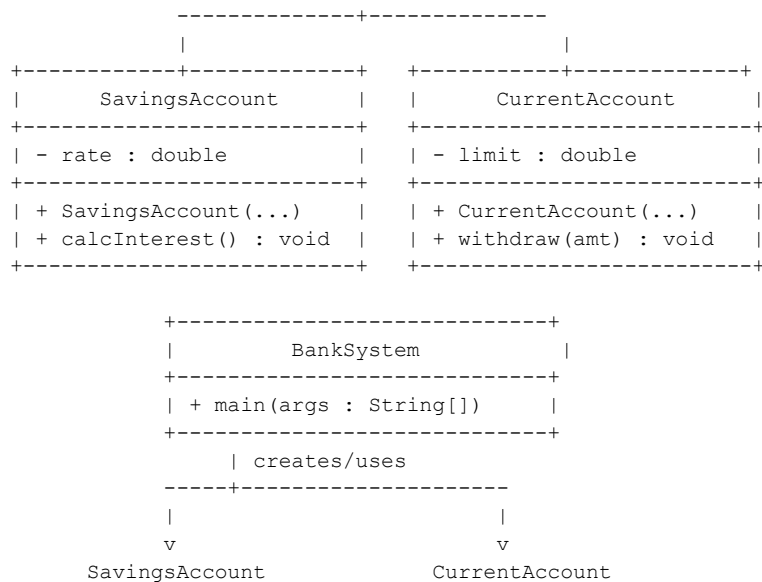
15. j) Otherwise display Wrong choice!.

16. Close Scanner.

17. Stop.

4. UML Class Diagram





UML relationship: SavingsAccount and CurrentAccount inherit from Account. BankSystem creates and uses objects of both derived classes.

5. Important Java Concepts Demonstrated

Concept	Where it appears
Inheritance	SavingsAccount extends Account; CurrentAccount extends Account.
super()	Derived constructors call the Account constructor using super(no, bal, type).
Method overriding	CurrentAccount provides its own withdraw(double amt) method.
Polymorphism	withdraw() behaves differently for SavingsAccount and CurrentAccount.
Constructor	Account, SavingsAccount and CurrentAccount initialize object data.
Object creation	new SavingsAccount(...) and new CurrentAccount(...).

6. Java Source Code

```

package javacore;

import java.util.Scanner;

// Base class for all accounts
class Account {
    int accNo;
    double balance;
    String type;

    Account(int no, double bal, String t) {
        accNo = no;
        balance = bal;
        type = t;
    }

    void deposit(double amt) {

```

```

        balance += amt;
        System.out.println("Deposited: " + amt);
        System.out.println("Balance: " + balance);
    }

    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("Withdrawn: " + amt);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Not enough balance!");
        }
    }

    void transfer(Account other, double amt) {
        if (amt <= balance) {
            balance -= amt;
            other.balance += amt;
            System.out.println("Transfer done!");
        } else {
            System.out.println("Transfer failed!");
        }
    }

    void show() {
        System.out.println("Acc No: " + accNo);
        System.out.println("Type : " + type);
        System.out.println("Bal : " + balance);
    }
}

// Savings account class
class SavingsAccount extends Account {
    double rate;

    SavingsAccount(int no, double bal, double r) {
        super(no, bal, "Savings");
        rate = r;
    }

    void calcInterest() {
        double interest = balance * rate / 100;
        System.out.println("Interest: " + interest);
    }
}

// Current account class
class CurrentAccount extends Account {
    double limit;

    CurrentAccount(int no, double bal, double l) {
        super(no, bal, "Current");
        limit = l;
    }

    @Override
    void withdraw(double amt) {
        if (amt <= balance + limit) {
            balance -= amt;
            System.out.println("Withdraw ok. Balance: " + balance);
        } else {
            System.out.println("Overdraft limit crossed!");
        }
    }
}

// Main class
public class BankSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        SavingsAccount sAcc = new SavingsAccount(101, 5000, 5);
        CurrentAccount cAcc = new CurrentAccount(201, 3000, 2000);
    }
}

```

```

int ch;
do {
    System.out.println("\n--- Bank Menu ---");
    System.out.println("1. Deposit (Savings)");
    System.out.println("2. Withdraw (Savings)");
    System.out.println("3. Interest (Savings)");
    System.out.println("4. Withdraw (Current)");
    System.out.println("5. Transfer Savings -> Current");
    System.out.println("6. Show Accounts");
    System.out.println("7. Exit");
    System.out.print("Choice: ");
    ch = sc.nextInt();

    switch (ch) {
        case 1:
            System.out.print("Amount: ");
            sAcc.deposit(sc.nextDouble());
            break;
        case 2:
            System.out.print("Amount: ");
            sAcc.withdraw(sc.nextDouble());
            break;
        case 3:
            sAcc.calcInterest();
            break;
        case 4:
            System.out.print("Amount: ");
            cAcc.withdraw(sc.nextDouble());
            break;
        case 5:
            System.out.print("Amount: ");
            sAcc.transfer(cAcc, sc.nextDouble());
            break;
        case 6:
            System.out.println("Savings:");
            sAcc.show();
            System.out.println("Current:");
            cAcc.show();
            break;
        case 7:
            System.out.println("Bye!");
            break;
        default:
            System.out.println("Wrong choice!");
    }
} while (ch != 7);

sc.close();
}

```